

Untitled

April 26, 2026

```
[7]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, log1p
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator
import matplotlib.pyplot as plt

spark = SparkSession.builder.appName("RevenueReviewsClustering").getOrCreate()

df = spark.read.csv("hdfs://localhost:9000/steam-data/games_cleaned.csv",
                    header=True, inferSchema=False,
                    quote='', escape='')

df = df.withColumn("Positive", col("Positive").cast(DoubleType()))
df = df.withColumn("Negative", col("Negative").cast(DoubleType()))
df = df.withColumn("estimated_revenue", col("estimated_revenue").
    ↪cast(DoubleType()))
df = df.withColumn("Price", col("Price").cast(DoubleType()))
df = df.withColumn("estimated_owners_mid", col("estimated_owners_mid").
    ↪cast(DoubleType()))

df = df.na.drop(subset=["Positive", "Negative", "estimated_revenue"])

df = df.filter((col("Positive") + col("Negative")) > 0)
df = df.filter(col("estimated_revenue") > 0)

df = df.withColumn("total_reviews", col("Positive") + col("Negative"))
df = df.withColumn("log_revenue", log1p(col("estimated_revenue")))
df = df.withColumn("log_reviews", log1p(col("total_reviews")))

assembler = VectorAssembler(inputCols=["log_revenue", "log_reviews"],
                             outputCol="raw_features")
df = assembler.transform(df)

scaler = StandardScaler() \
    .setInputCol("raw_features") \
```

```

        .setOutputCol("features") \
        .setWithMean(True) \
        .setWithStd(True)
scaler_model = scaler.fit(df)
df = scaler_model.transform(df)

evaluator = ClusteringEvaluator() \
    .setFeaturesCol("features") \
    .setMetricName("silhouette")

scores = []
for k in range(2, 8):
    kmeans = KMeans() \
        .setK(k) \
        .setSeed(42) \
        .setFeaturesCol("features") \
        .setPredictionCol("prediction") \
        .setMaxIter(20)
    model = kmeans.fit(df)
    predictions = model.transform(df)
    silhouette = evaluator.evaluate(predictions)
    scores.append((k, silhouette))
    print("k=" + str(k) + ", silhouette=" + str(round(silhouette, 4)))

best_k = max(scores, key=lambda x: x[1])[0]
print("\nBest k=" + str(best_k))

ks = [s[0] for s in scores]
sils = [s[1] for s in scores]
plt.figure(figsize=(8, 4))
plt.plot(ks, sils, marker='o')
plt.xlabel("k")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs Number of Clusters")
plt.tight_layout()
plt.savefig("silhouette_scores.png", dpi=120)
plt.show()

# train the final model with best k
kmeans = KMeans() \
    .setK(best_k) \
    .setSeed(42) \
    .setFeaturesCol("features") \
    .setPredictionCol("prediction") \
    .setMaxIter(20)
model = kmeans.fit(df)

```

```

predictions = model.transform(df)

silhouette = evaluator.evaluate(predictions)
print("Silhouette with squared euclidean distance = " + str(silhouette))

centers = model.clusterCenters()
print("\nCluster Centers: ")
for center in centers:
    print(center)

print("\nCluster Sizes: " + str(model.summary.clusterSizes))

summary = predictions.groupby("prediction").agg(
    {"estimated_revenue": "mean", "total_reviews": "mean", "AppID": "count"}
).withColumnRenamed("avg(estimated_revenue)", "avg_revenue") \
  .withColumnRenamed("avg(total_reviews)", "avg_reviews") \
  .withColumnRenamed("count(AppID)", "num_games") \
  .orderBy("prediction")
print("\nCluster Summary:")
summary.show()

corr = predictions.stat.corr("log_revenue", "log_reviews")
print("Pearson correlation (log scale): " + str(round(corr, 4)))

for c in range(best_k):
    subset = predictions.filter(col("prediction") == c)
    c_corr = subset.stat.corr("log_revenue", "log_reviews")
    count = subset.count()
    print("Cluster " + str(c) + ": correlation=" + str(round(c_corr, 4)) + ",
    ↪games=" + str(count))

plot_df = predictions.select("log_revenue", "log_reviews", "prediction").
    ↪toPandas()

plt.figure(figsize=(10, 6))
for c in range(best_k):
    sub = plot_df[plot_df["prediction"] == c]
    plt.scatter(sub["log_reviews"], sub["log_revenue"],
                label="Cluster " + str(c) + " (" + str(len(sub)) + " games)",
                alpha=0.4, s=8)
plt.xlabel("log(1 + total reviews)")
plt.ylabel("log(1 + estimated revenue)")
plt.title("KMeans Clustering (k=" + str(best_k) + "): Revenue vs Number of
    ↪Reviews")
plt.legend()
plt.tight_layout()
plt.savefig("cluster_revenue_reviews.png", dpi=120)

```

```

plt.show()
# try clustering without log transform for comparison
assembler_raw = VectorAssembler(inputCols=["estimated_revenue", "total_reviews"],
                                outputCol="raw_features_nolog")
df_nolog = assembler_raw.transform(df)

scaler_nolog = StandardScaler() \
    .setInputCol("raw_features_nolog") \
    .setOutputCol("features_nolog") \
    .setWithMean(True) \
    .setWithStd(True)
df_nolog = scaler_nolog.fit(df_nolog).transform(df_nolog)

kmeans_nolog = KMeans() \
    .setK(2) \
    .setSeed(42) \
    .setFeaturesCol("features_nolog") \
    .setPredictionCol("prediction_nolog") \
    .setMaxIter(20)
model_nolog = kmeans_nolog.fit(df_nolog)
preds_nolog = model_nolog.transform(df_nolog)

eval_nolog = ClusteringEvaluator() \
    .setFeaturesCol("features_nolog") \
    .setPredictionCol("prediction_nolog") \
    .setMetricName("silhouette")
sil_nolog = eval_nolog.evaluate(preds_nolog)
print("Silhouette WITHOUT log: " + str(round(sil_nolog, 4)))

summary_nolog = preds_nolog.groupBy("prediction_nolog").agg(
    {"estimated_revenue": "mean", "total_reviews": "mean", "AppID": "count"}
).orderBy("prediction_nolog")
summary_nolog.show()
spark.stop()

```

k=2, silhouette=0.74

k=3, silhouette=0.5508

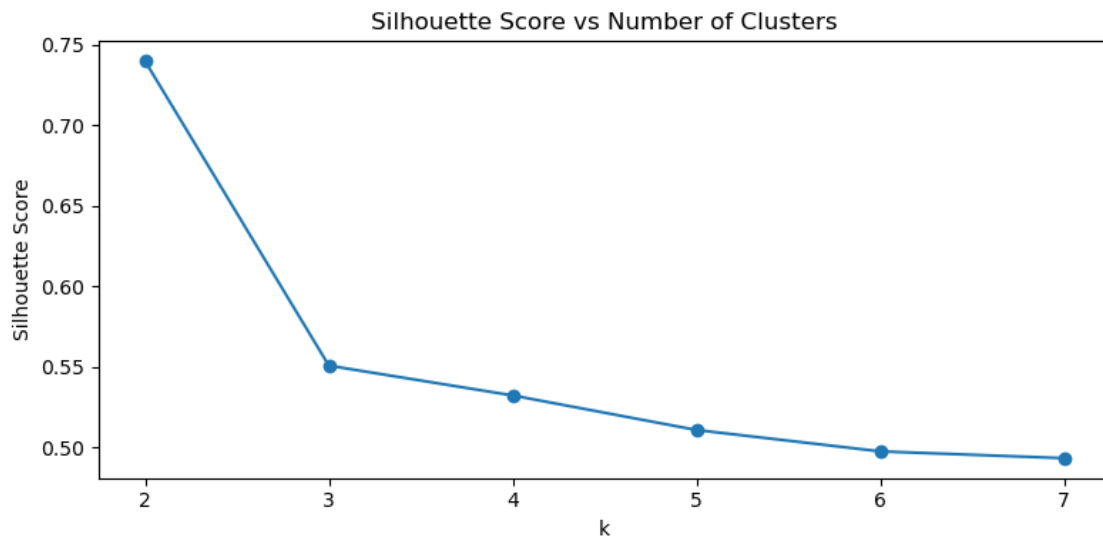
k=4, silhouette=0.5323

k=5, silhouette=0.5108

k=6, silhouette=0.4975

k=7, silhouette=0.4933

Best k=2



Silhouette with squared euclidean distance = 0.7399876188518941

Cluster Centers:

[-0.35639251 -0.38151438]

[1.39075034 1.48878341]

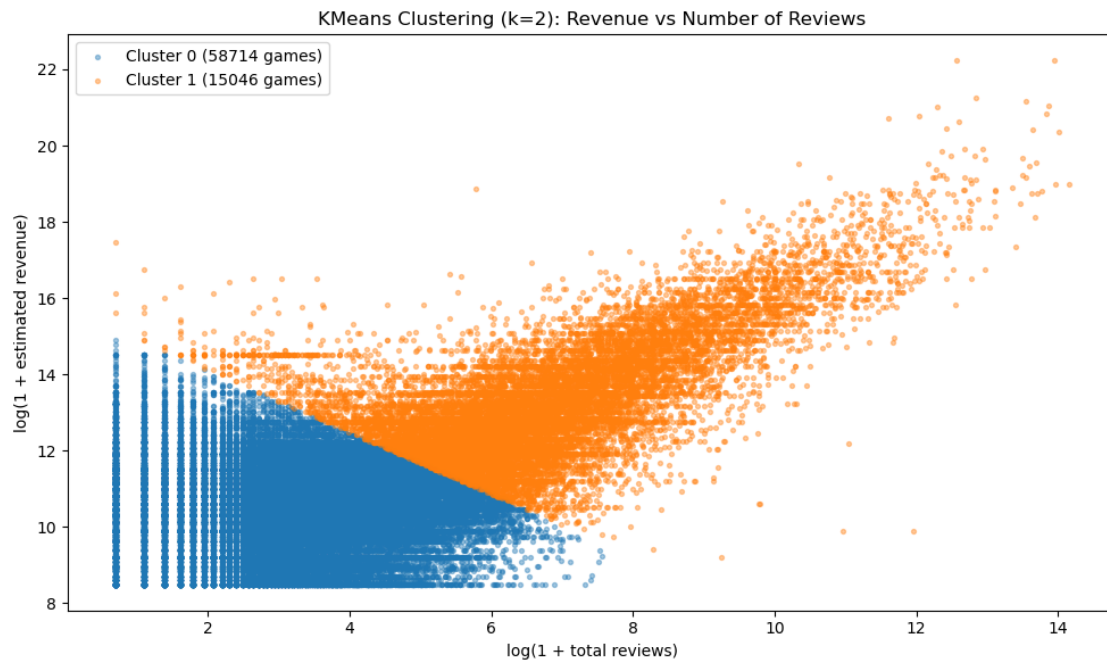
Cluster Sizes: [58714, 15046]

Cluster Summary:

prediction	num_games	avg_revenue	avg_reviews
0	58714	59871.49913138263	37.184385325476036
1	15046	4610796.553901369	6975.732952279675

Pearson correlation (log scale): 0.6239

Cluster 0: correlation=-0.0212, games=58714
Cluster 1: correlation=0.6348, games=15046



Silhouette WITHOUT log: 0.9999

+-----+-----+-----+-----+				
prediction_nolog	count(AppID)	avg(estimated_revenue)	avg(total_reviews)	
+-----+-----+-----+-----+				
	0	73758	866222.5134900621	1433.1075408769218
	1	2	4.49925E9	718488.0
+-----+-----+-----+-----+				

[]: